

Lab 05: Factor analysis 2

Author: **N.J. de Winter** (n.j.de.winter@vu.nl)

Assitant Professor Vrije Universiteit Amsterdam

Statistics and Data Analysis Course

Learning goals:

- Apply and improve your knowledge of Python and Jupyter
- Continue practicing with factor analysis
 - Learn about data pretreatment (scaling and normalization)
 - Apply principle component analysis on time series
 - Interpret PCA results in a real-world example to learn more about relationships within a dataset
- Develop a feeling for how statistical tools can help you, but you still require *your interpretation* to draw conclusions.

Introduction

In the last lab, you have become familiar with **Principle Component Analysis** and have learned to interpret the *scores*, *loadings* and *eigenvalues* that define the outcome of a PCA. In this lab, we will practice a bit more with this data analysis method and apply it on a realistic time series to discover how we can use PCA as a tool in the Earth Sciences.

As always, let's start by loading the packages we will need.

Exercise 1: In the code box below, load the packages `numpy`, `pyplot` and `preprocessing` (found in the `sklearn` package) and import the functions `PCA` (from `sklearn.decomposition`) and `loadmat` (from `scipy.io`).

In []:

The latter of these functions we will need to load the data for this lab. This dataset (`lab05.mat`) is provided as a Matlab table (`.mat` -file) and requires a special function to load:

In []:

```
lab05 = loadmat('lab05.mat')
```

The `lab05` file contains data on drainage (m^3/s), water temperature ($^{\circ}C$), and dissolved chlorophyll ($\mu g/l$), nitrogen (mg/l) and phosphate (mg/l) in the Rhine at Lobith, where the Rhine enters the Netherlands, starting from 1989.

When loading a `.mat` file `scipy.io` returns a dictionary with variable names as keys, and matrices as values. The data matrix contains all these variables (column 1: temperature, column 2: drainage, column 3: chlorophyll, column 4: nitrogen, column 5: phosphate). You can for example print this matrix using the following command:

In []:

```
print(lab05['data'])
```

All measurements are ordered according to time of observation. The vector `T`, in the dictionary `lab05`, gives the day since January 1, 1989.

Chlorophyll are the pigments in green plants that capture energy/light for photosynthesis. Chlorophyll in water relates to the concentration of algae. Algae are more abundant in summer, when temperatures are warmer. Algae need nitrogen and phosphate for their growth. So, when there are lots of algae the concentrations of N and P are reduced. The main source of N and P in river water is polluted water from households and industries. Because of that, N and P concentrations may be influenced by changes in environmental laws and restrictions. These have more influence on N concentrations, than P concentrations because the latter are difficult to remove from polluted water. Water temperature and drainage are strongly seasonally dependent. In summer, temperature is high and drainage low, while the opposite occurs in winter.

Question 1: After reading the information about the variables and their units in this dataset above, can you think of a potential issue we will encounter when combining all these variables in one data analysis?

Answer 1:

Before we apply a PCA, we need to standardize our dataset so that all variables receive equal weight in the PCA. You can do this using the following commands:

In []:

```
scaler = preprocessing.StandardScaler() # Define the preprocessing function we will apply to our data
standardized_data = scaler.fit_transform(lab05['data']) # Apply the preprocessing on our dataset
```

Note that the syntax we use here to do our preprocessing is very similar to how we ran our PCA in Lab04 (have a look back at that lab in case you forgot!): We first define a preprocessing routine and then apply that routine on our data using the `.fit_transform()` -syntax.

Question 2: What would happen in the PCA if you would not standardize your dataset?

Answer 2:

Exercise 2: In lab 04, we already implemented a PCA. Have a look in lab 04 which commands we used to execute a PCA. Then apply these commands to the standardized data matrix. Don't forget to extract the *scores*, *loadings* and *eigenvalues* from the result.

In []:

Exercise 3: Plot the percentage of variance explained by each principle component.

In []:

I think you would agree that this result looks quite a bit different from the result we obtained in **Lab04**!

Question 3: Describe the main difference you observe with respect to the dataset in Lab04. What does this result tell you about this dataset compared to the one we used before?

Answer 3:

Question 4: How many principal components would you withhold in your analysis and why?

Answer 4:

Let's have a look at which variables strongly influence the first three principal components to try to make an interpretation about what these principal components may represent.

Question 5: Which outcome of the PCA do we need to consider to check which variables are important for which principle components?

Answer 5:

Plotting this outcome may help. Below is an example for the first principal component:

In []:

```
plt.figure(2)
plt.bar(np.arange(1, 6, 1), loadings[:, 0]) # Create a bar chart with numbers of variables on the horizontal axis and loadings on the vertical axis
plt.axhline(color = 'black') # Draw a black horizontal line indicating a loading of zero.
plt.xlim([0, 6]) # Set the limits on the horizontal axis
plt.ylim([-0.5, 1]) # Set the limits on the vertical axis
plt.xticks(np.arange(1, 6, 1), ['temperature', 'drainage', 'chlorophyll', 'N', 'P']) # Add names for the variables
plt.title('PC1') # Provide a title
plt.xlabel('Variables') # Provide horizontal axis title
plt.ylabel('PC loading') # Provide vertical axis title
```

Exercise 4: Make the same plot for the other principal components.

In []:

Question 6: Based on the plot above, what do you think is the physical meaning of the first three principle components?

Answer 6:

The goal of principal component analysis is to transform several correlated variables into fewer uncorrelated principal components (that maximize variance).

Question 7: How can we check if we succeeded in doing this?

Answer 7:

Exercise 5: Use a function that you used in **Lab03** and **Lab04** to check for correlation between variables on the original data, the preprocessed data and on the principle components (the "transformed data") to check if our PCA did what it was supposed to do.

In []:

Question 7: Do the lots above show the outcome you expect?

Answer 7:

To find out what our principle components mean, let's now have a look at time series plots of the principal component scores from the first three principal components. To compare time series and interpret results, it might be better to use subplots:

```
In [ ]: plt.figure(1)
plt.subplot(311)
plt.plot(lab05['T'], scores[:, 0])
plt.title('PC1')
plt.ylabel('PC score')
plt.subplot(312)
plt.plot(lab05['T'], scores[:, 1])
plt.title('PC2')
plt.ylabel('PC score')
plt.subplot(313)
plt.plot(lab05['T'], scores[:, 2])
plt.title('PC3')
plt.xlabel('Days since January 1, 1989')
plt.ylabel('PC score')
```

Question 8: Based on the plots you just created, do you still agree with the interpretation of the principle components you made in your answer to **Question 6**?

Answer 8: